

Feature selection performance assessment

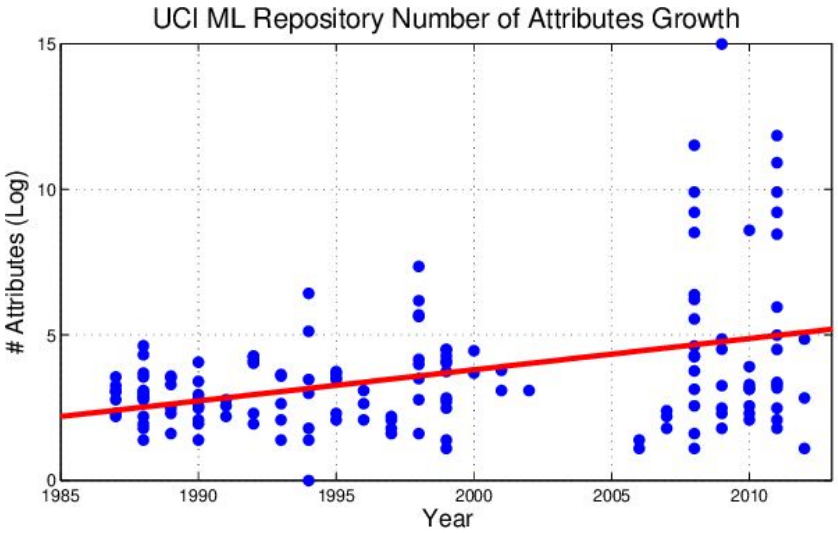
Jeroen Overschie
Supervisors: G. Azzopardi & A.M.J.A. Alsahaf

Research Internship 2020
MSc Data Science & Systems Complexity

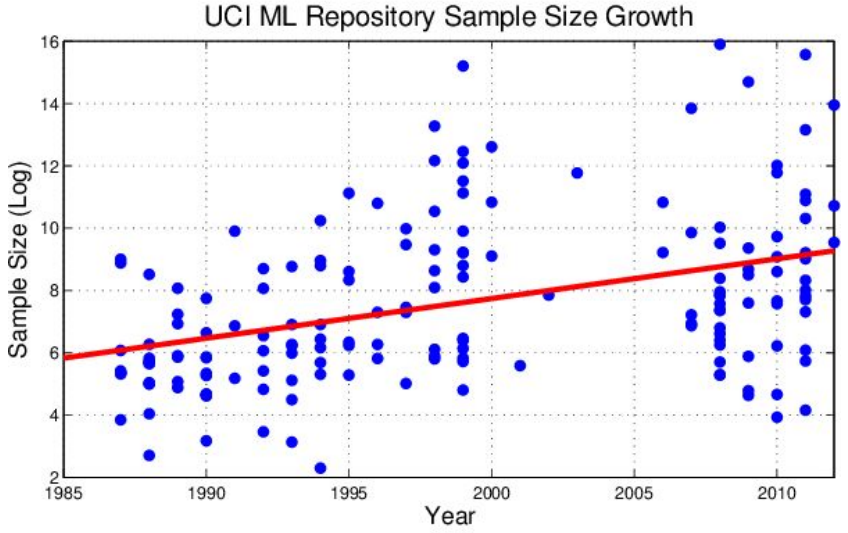
Why?

- Why bother with feature selection evaluation?

Datasets are increasing in size



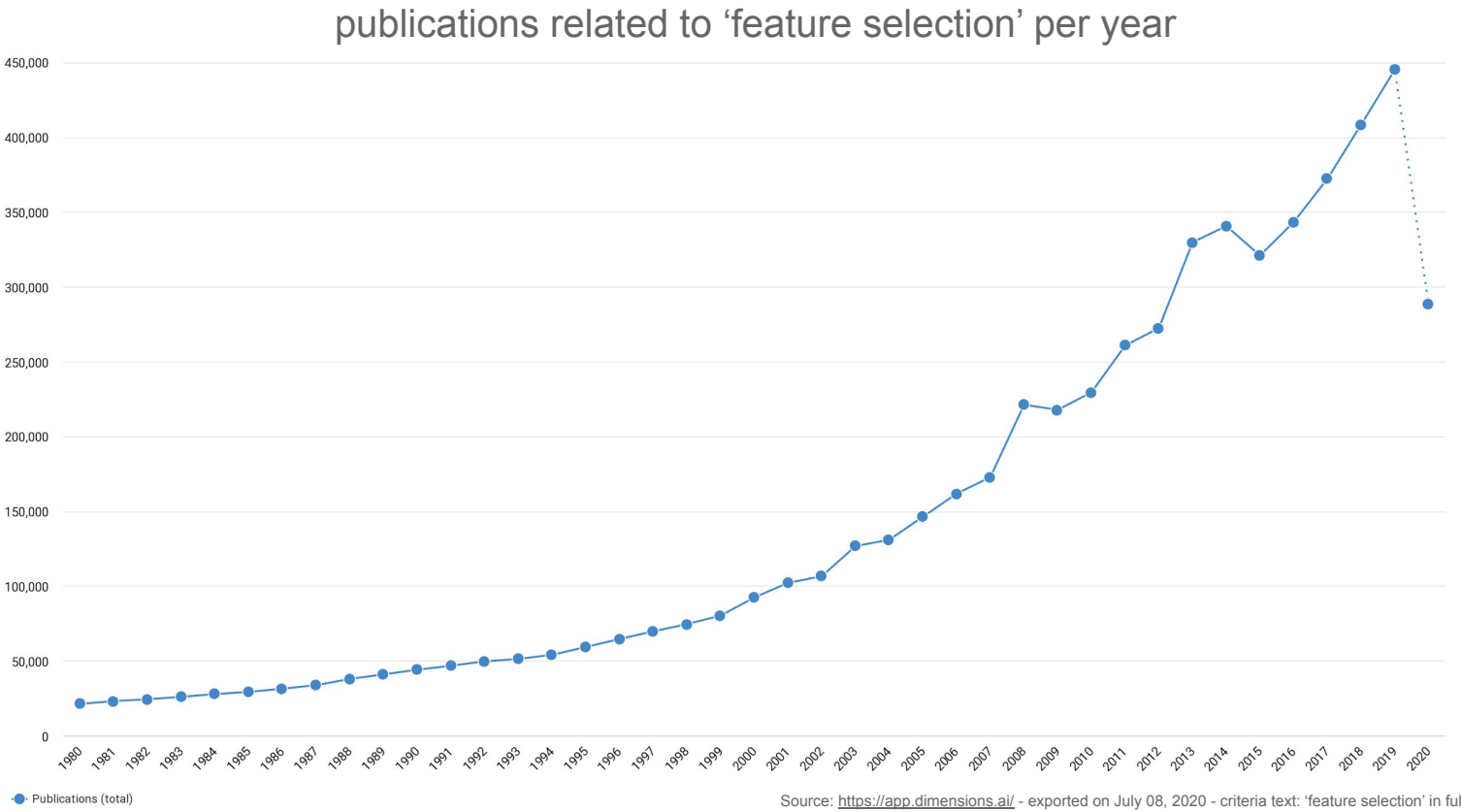
(a) feature number growth



(b) sample size growth

Source: Li, J., & Liu, H. (2017). Challenges of Feature Selection for Big Data Analytics. *IEEE Intelligent Systems, 32*, 9-15.

… and so is the literature on **feature selection**



Source: https://app.dimensions.ai/ - exported on July 08, 2020 - criteria text: ‘feature selection’ in full data.

However, papers use a wide variety of evaluation methods.

e.g. common validation classifiers:

supervised: *k-NN*, *SVM*, *Decision Tree*, *Naive Bayes* with `classif. acc`
unsupervised: *k-means* with `ACC` and `NMI`
And few take advantage of *a priori* knowledge.

We desire a universal framework for feature selection method evaluation.

Presentation setup



Contents

✓ Why feature selection evaluation?

2. Meaningful evaluation
   a. Relevant criteria
   b. A pipeline

3. `fseval` under the hood
   a. application interface
   b. multi-core processing
   c. analysis

4. (Experiment)

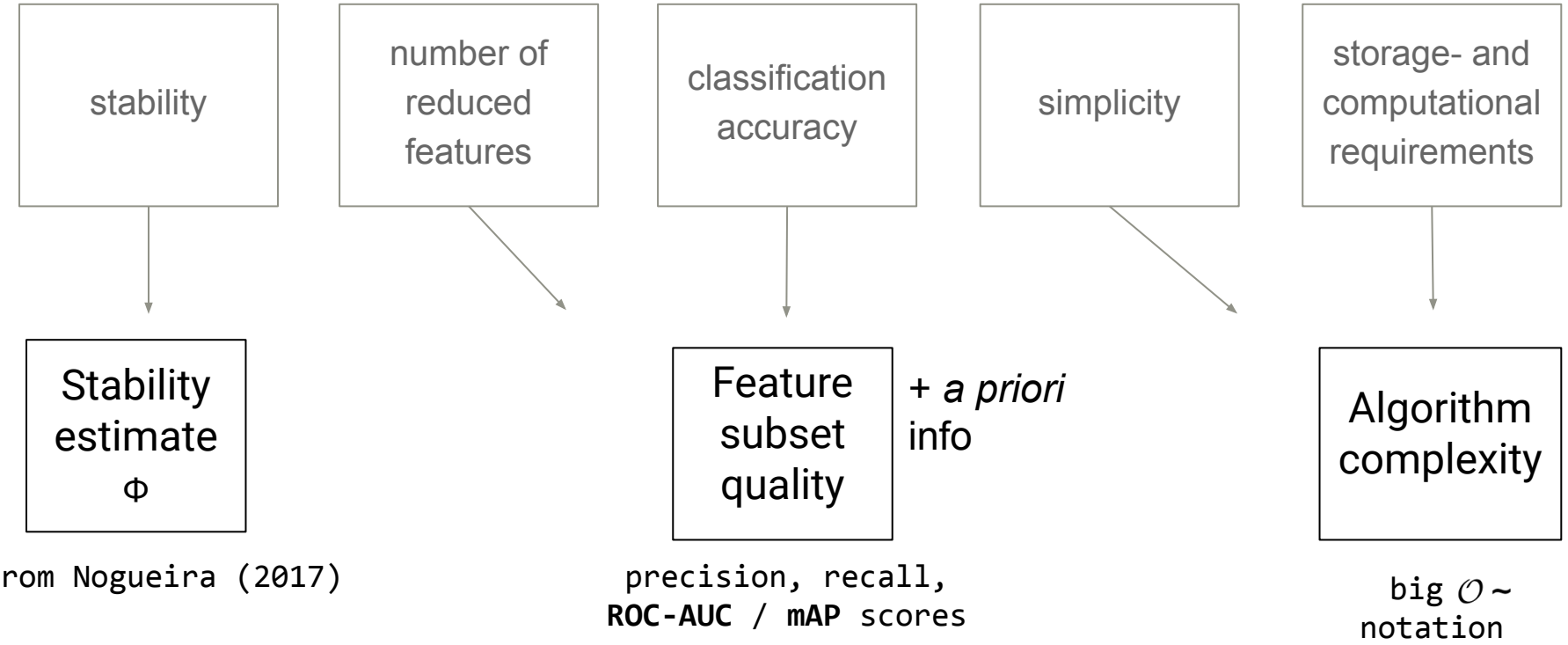
5. Future work

Meaningful evaluation

Which criteria are relevant?

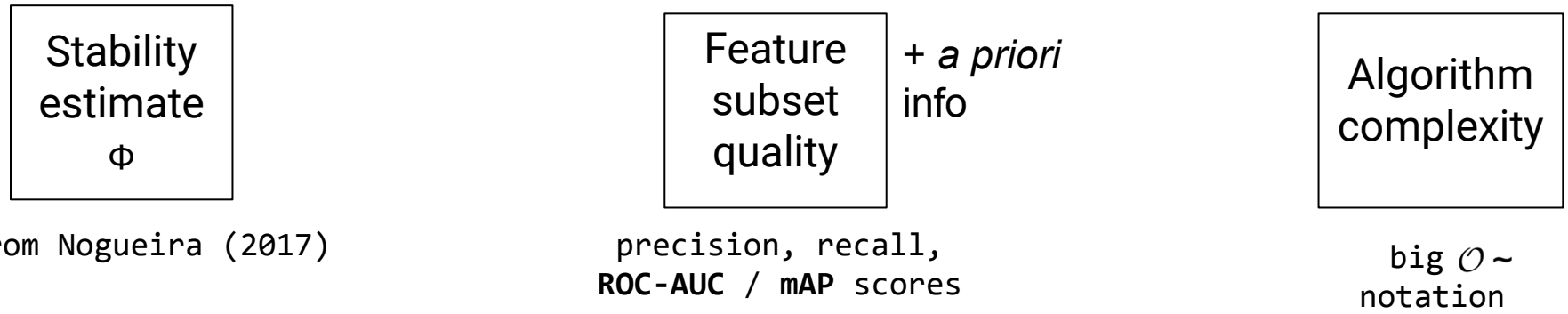
Meaningful evaluation → relevant criteria

Chandrashekar (2013):

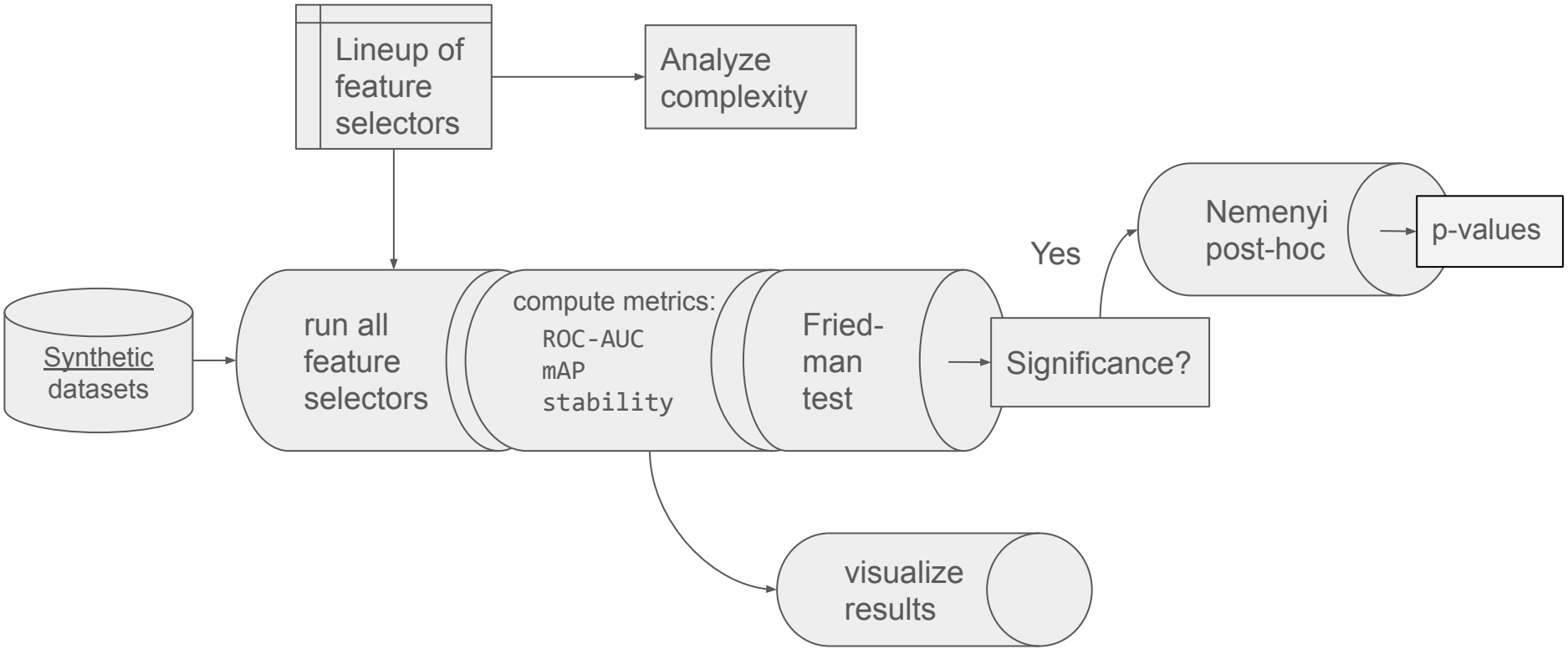


Meaningful evaluation

So, given these new metrics - how would a benchmark pipeline look like?



Meaningful evaluation → pipeline



`fseval` under the hood

→ An implementation of the pipeline ?

`fseval` under the hood

Define
feature selectors

Define
datasets using '*dataset descriptor*' .csv file

chi2.py

```python
from ..pipeline.compute_pool import run_pool,
ranking_pool
from sklearn.feature_selection import chi2
import numpy as np

def chi2_ranking(X, y):
    scores, _ = chi2(X, y)
    ranking = np.argsort(-scores)
    return ranking

if __name__ == '__main__':
    run_pool(ranking_pool, chi2_ranking, 'Chi2')
```

descriptor.csv

adapter	dir	path	replica_no	replicas	name	p	n	descr	p_informative
rebate	./e	./example/replica1.gz	1	2	3-way Epi	20	1600	3-way	17,18,19
rebate	./e	./example/replica2.gz	2	2	3-way Epi	20	1600	3-way	17,18,19

replicas

ground truth

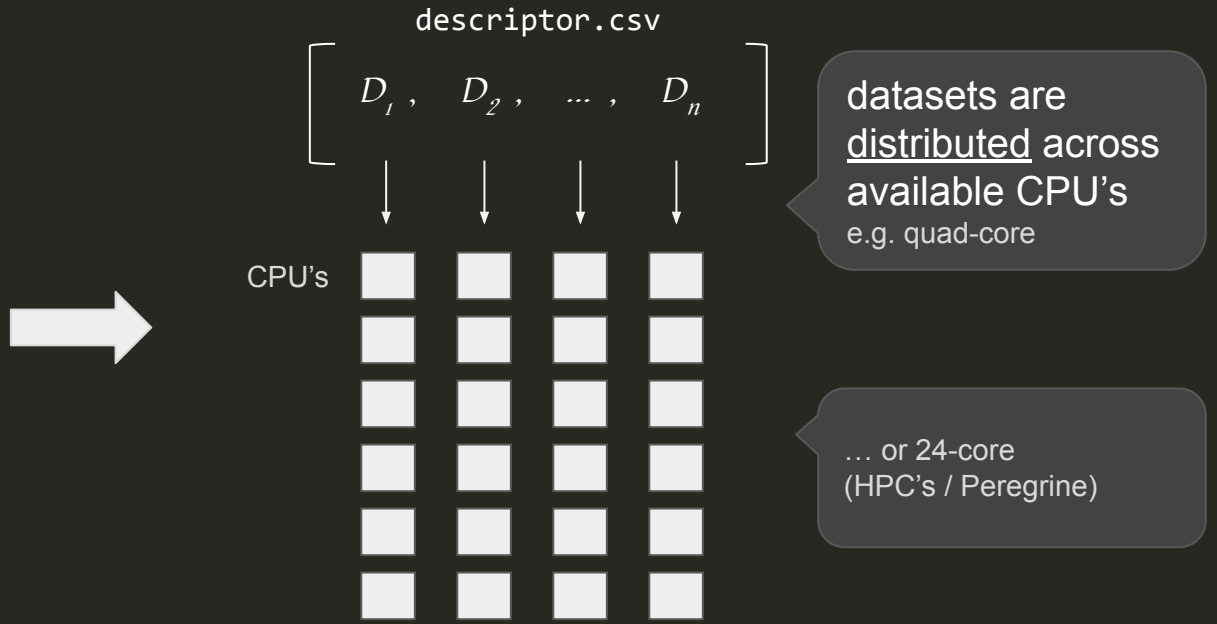
dataset agnostic
(looks for like-named module
in fseval.adapters)

`fseval` under the hood

Run
benchmark

TERMINAL

```
$ python -m chi2
descriptor.csv
--batch_id=test
```



`fseval` under the hood

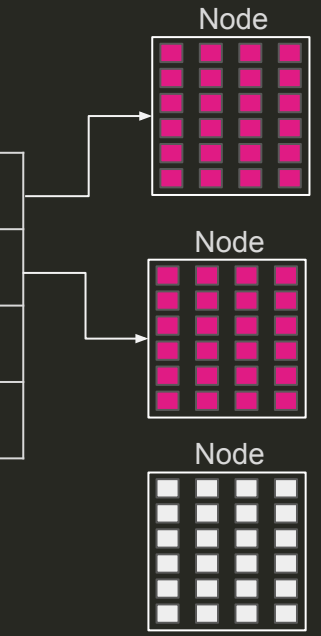
Run
benchmark

SLURM environments are supported — using *job arrays*.
e.g. Peregrine is a SLURM type cluster

`descriptor.csv`

Construct
job array

[24 datasets]	job 1.0	RUNNING
[24 datasets]	job 1.1	RUNNING
[24 datasets]	job 1.2	WAITING
[24 datasets]	job 1.3	WAITING



High CPU usage efficiency (> 90%) if all 24 are replicas of same dataset.

`fseval` under the hood

Run
benchmark

CLI tool
→ used to easily submit jobs to Peregrine

allows for allocating datasets to varying amounts of CPU's or nodes.



`fseval` under the hood

Analyse results

Results are stored in `.csv` files

results/test/
ranking/replica1.gz.csv

	feature_index	feature_rank	subset	cpu_ti	ranking_method
	6	0	6	0.01	Chi2
	15	1	6,15	0.01	Chi2
	19	2	6,15,19	0.01	Chi2
	8	3	6,8,15,19	0.01	Chi2
	14	4	6,8,14,15,19	0.01	Chi2
	17	5	6,8,14,15,17,1	0.01	Chi2
	13	6	6,8,13,14,15,1	0.01	Chi2
	7	7	6,7,8,13,14,15	0.01	Chi2
	2	8	2,6,7,8,13,14,	0.01	Chi2
	18	9	2,6,7,8,13,14,	0.01	Chi2
	9	10	2,6,7,8,9,13,1	0.01	Chi2
	12	11	2,6,7,8,9,12,1	0.01	Chi2
	4	12	2,4,6,7,8,9,12	0.01	Chi2
	10	13	2,4,6,7,8,9,10	0.01	Chi2
	5	14	2,4,5,6,7,8,9,	0.01	Chi2
	3	15	2,3,4,5,6,7,8,	0.01	Chi2
	16	16	2,3,4,5,6,7,8,	0.01	Chi2
	11	17	2,3,4,5,6,7,8,	0.01	Chi2
	0	18	0,2,3,4,5,6,7,	0.01	Chi2
	1	19	0,1,2,3,4,5,6,	0.01	Chi2

Analyse using ground truth

TERMINAL

```
$ python -m
fseval.analysis.roc
descriptor.csv
--batch_id=test
```

results/test/
roc/replica1.gz.csv

	feature_rank	rankir	recall	precision	fpr
	0	Chi2	0	0	0.05
	1	Chi2	0	0	0.1
	2	Chi2	0	0	0.15
	3	Chi2	0	0	0.2
	4	Chi2	0	0	0.25
	5	Chi2	0	0	0.3
	6	Chi2	0.33	0.14	0.32
	7	Chi2	0.33	0.13	0.37
	8	Chi2	0.33	0.11	0.42
	9	Chi2	0.33	0.1	0.47
	10	Chi2	0.33	0.09	0.53
	11	Chi2	0.33	0.08	0.58
	12	Chi2	0.67	0.15	0.61
	13	Chi2	0.67	0.14	0.67
	14	Chi2	0.67	0.13	0.72
	15	Chi2	0.67	0.13	0.78
	16	Chi2	1	0.18	0.82
	17	Chi2	1	0.17	0.88
	18	Chi2	1	0.16	0.94
	19	Chi2	1	0.15	1

`fseval` under the hood

Analyse results

Results are stored in `.csv` files

results/test/ranking/replica1.gz.csv

	feature_index	feature_rank	subset	cpu_ti	ranking_method
	6	0	6	0.01	Chi2
	15	1	6,15	0.01	Chi2
	19	2	6,15,19	0.01	Chi2
	8	3	6,8,15,19	0.01	Chi2
	14	4	6,8,14,15,19	0.01	Chi2
	17	5	6,8,14,15,17,1	0.01	Chi2
	13	6	6,8,13,14,15,1	0.01	Chi2
	7	7	6,7,8,13,14,15	0.01	Chi2
	2	8	2,6,7,8,13,14,	0.01	Chi2
	18	9	2,6,7,8,13,14,	0.01	Chi2
	9	10	2,6,7,8,9,13,1	0.01	Chi2
	12	11	2,6,7,8,9,12,1	0.01	Chi2
	4	12	2,4,6,7,8,9,12	0.01	Chi2
	10	13	2,4,6,7,8,9,10	0.01	Chi2
	5	14	2,4,5,6,7,8,9,	0.01	Chi2
	3	15	2,3,4,5,6,7,8,	0.01	Chi2
	16	16	2,3,4,5,6,7,8,	0.01	Chi2
	11	17	2,3,4,5,6,7,8,	0.01	Chi2
	0	18	0,2,3,4,5,6,7,	0.01	Chi2
	1	19	0,1,2,3,4,5,6,	0.01	Chi2

Or other analyses:

Classif. accuracy

results/test/validation/replica1.gz.csv

	fold_nu	score	n_features	ranking	cpu_t	validation_method
	1	0.34	1	Chi2	0.1	KNN_5Fold
	1	0.32	2	Chi2	0.1	KNN_5Fold
	1	0.39	3	Chi2	0.09	KNN_5Fold
	1	0.43	4	Chi2	0.1	KNN_5Fold
	1	0.39	5	Chi2	0.15	KNN_5Fold
	1	0.35	6	Chi2	0.11	KNN_5Fold
	1	0.39	7	Chi2	0.13	KNN_5Fold
	1	0.41	8	Chi2	0.12	KNN_5Fold
	1	0.39	9	Chi2	0.13	KNN_5Fold
	1	0.84	10	Chi2	0.17	KNN_5Fold
	1	0.8	11	Chi2	0.14	KNN_5Fold
	1	0.73	12	Chi2	0.15	KNN_5Fold
	1	0.68	13	Chi2	0.14	KNN_5Fold
	1	0.68	14	Chi2	0.16	KNN_5Fold
	1	0.65	15	Chi2	0.15	KNN_5Fold
	1	0.64	16	Chi2	0.16	KNN_5Fold
	1	0.63	17	Chi2	0.17	KNN_5Fold
	1	0.6	18	Chi2	0.17	KNN_5Fold
	1	0.59	19	Chi2	0.16	KNN_5Fold
	1	0.58	20	Chi2	0.16	KNN_5Fold

`fseval` under the hood

Analyse
results

Pipeline steps can be chained together in a single SLURM job

job-array.sh

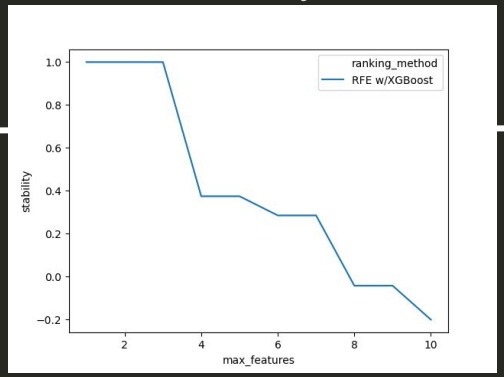
```
#SBATCH --time=05:00:00
#SBATCH --mem=10000
#SBATCH --partition=regular

module load Python/3.6.4-foss-2018a
python -m fseval.ranking.chi2            $data_collection --batch_id=test
python -m fseval.validation.knn                $data_collection
--batch_id=test
python -m fseval.analysis.roc            $data_collection --batch_id=test
python -m fseval.analysis.binary_subsets  $data_collection --batch_id=test
```

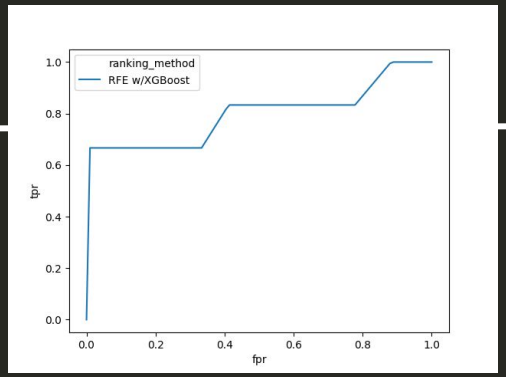
`fseval` under the hood

Plot results ✨

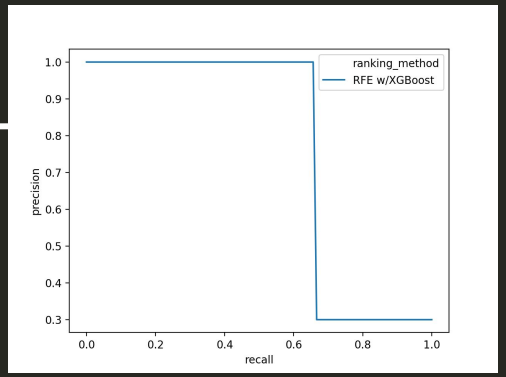
Stability

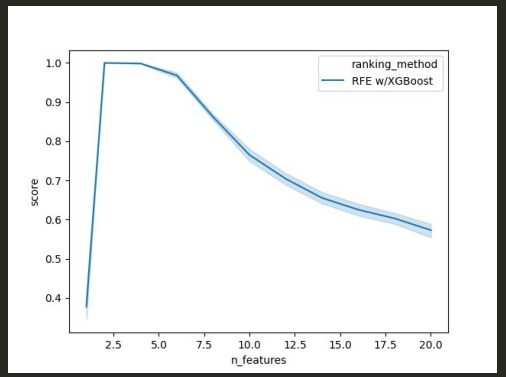


ROC



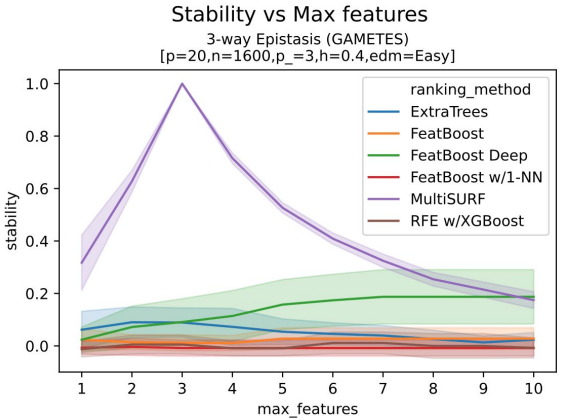
PR

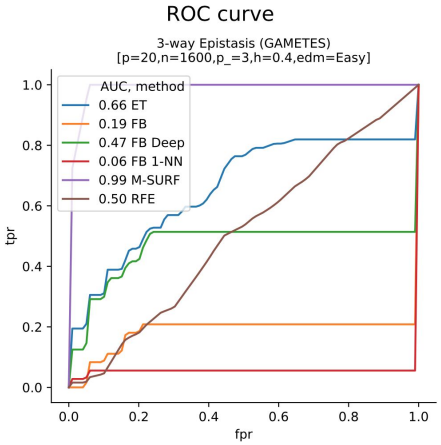


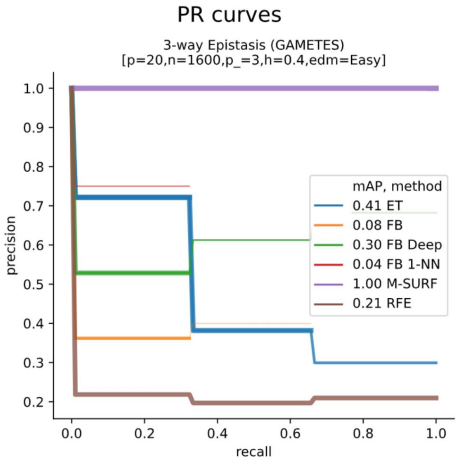


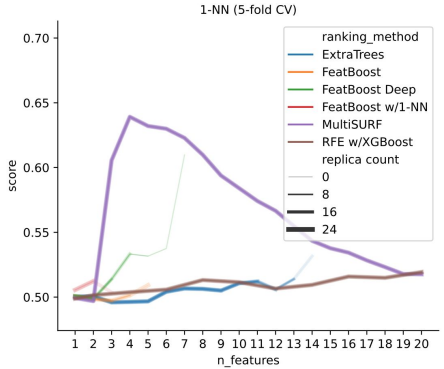
k-NN validation

Experiment









k-NN validation

Future work

Limitations & Ideas 💡

Future work

Limitations & Ideas 💡

Limitations:

- Metric set
  - elaborate on **scalability** and algorithm **complexity**
- Experiment
  - also analyse algorithm **storage complexity** and **simplicity**
  - more considerate **ranking/subset comparison**
  - **higher dimensionality** datasets

Future work

Limitations & **Ideas** 💡

Ideas:

- ‘Weighted’ graphical PR curve
  - lower the curve where less replicas reached that level
  - might provide better visual cues than line width
- Non-linear recall scheme
  - the success of a prediction task might not scale **linearly** with more informative features selected
  - reward an algorithm **larger portions of points** the more informative features are selected

More ideas on `fseval` specifically at: https://github.com/dunnkers/fseval/projects/1



Thanks! ✌🏻

university of groningen / faculty of science and engineering / computing science

Feature selection performance assessment

June 2020

MSc Research Internship INMSTAG-08
Student: J.G.S. Overschie
First supervisor: dr. G. Azzopardi
Second assessor: A.M.J.A. Alsahaf